

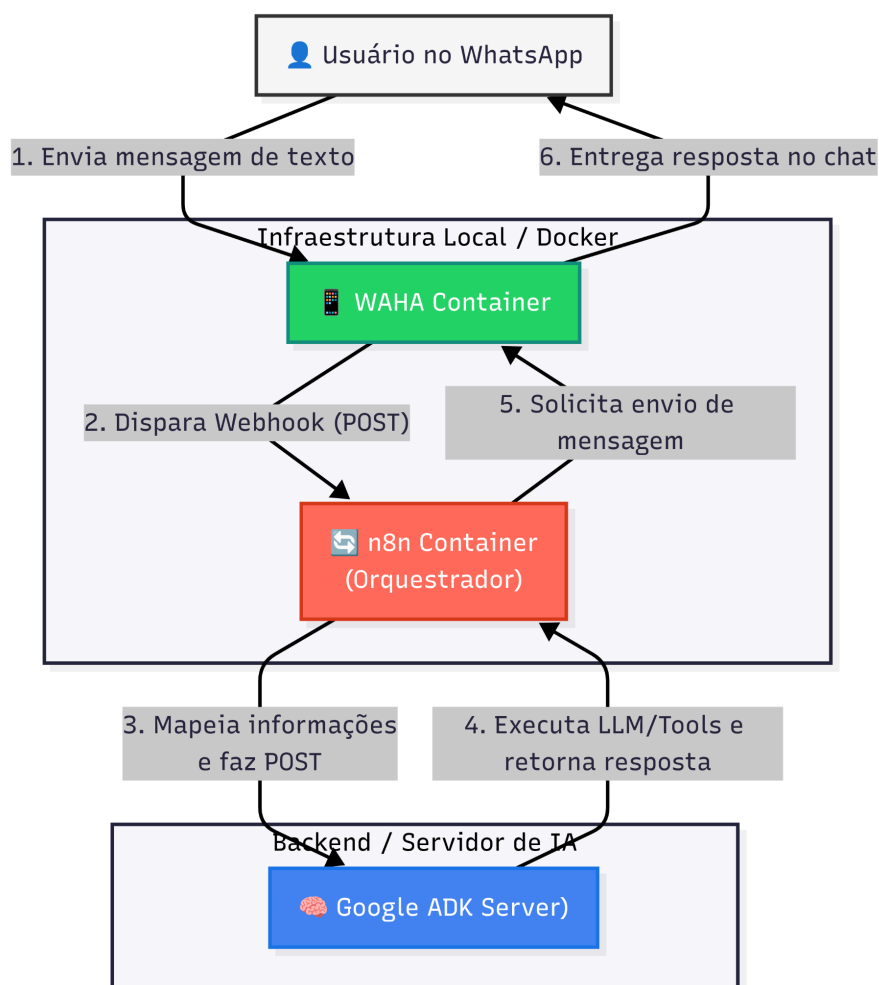
Relatório 11 - Prática: Criando agentes com n8n, ADK e Whatsapp (IV)

Lucas Augusto Nunes de Barros

Descrição da atividade

1. Configuração da Infraestrutura via Docker Compose

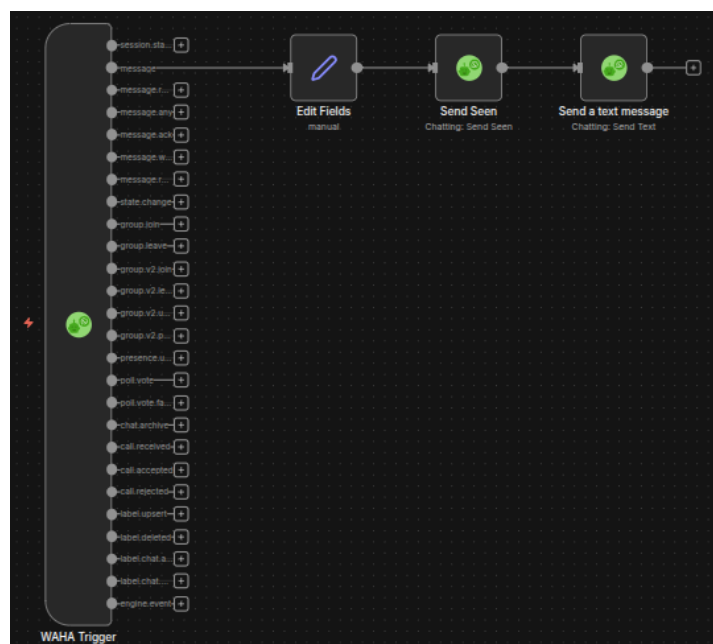
Nesta etapa prática, foi proposto o desenvolvimento de um agente local de automação de *WhatsApp*, integrando a API HTTP do *WAHA* (*WhatsApp HTTP API*) à plataforma de automação *n8n*. A infraestrutura foi inteiramente descrita em um arquivo *docker-compose.yml*, tornando a inicialização do ambiente fácil e reproduzível com um único comando. Os dois serviços (*n8n* e *waha*) foram colocados na mesma rede do Docker Compose, permitindo que se comuniquem pelo nome do serviço.



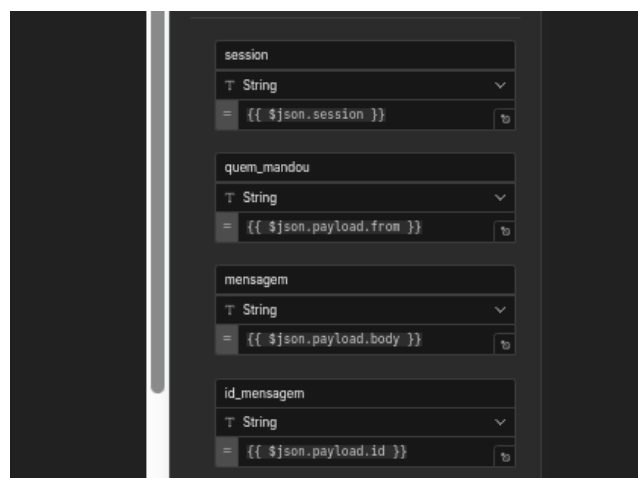
Arquitetura Proposta

2. Construção do Workflow de Automação (Ping-Pong)

Após a conexão com o *WhatsApp* através do *QR Code* no *dashboard* do *WAHA*, o fluxo *n8n* foi construído com quatro nós: um nó *WAHA Trigger* que recebe, via *webhook*, cada mensagem chegando na sessão conectada; um nó *Edit Fields* que extrai informações relevantes do *payload* recebido ; um nó *WAHA (Send Seen)*, que marca a mensagem como lida; e por último um nó *WAHA (Send Text)*, que responde automaticamente ecoando a mensagem recebida. O fluxo foi validado enviando uma mensagem de um número externo para o número conectado no *WAHA* e retornando automaticamente a confirmação de leitura e a resposta de texto, sem qualquer intervenção manual.



Fluxo de Automação no n8n

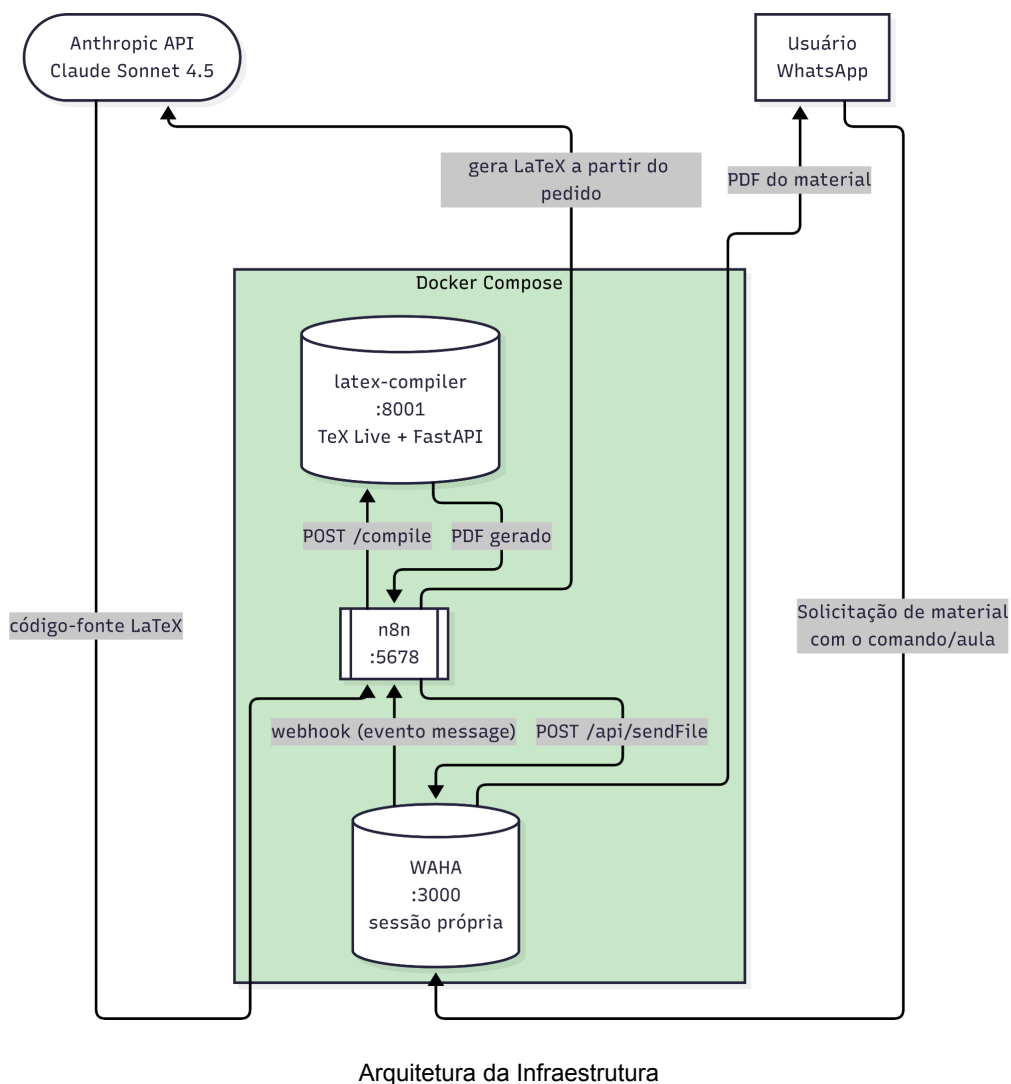


Dados Extraídos da Mensagem pelo Node Edit Fields

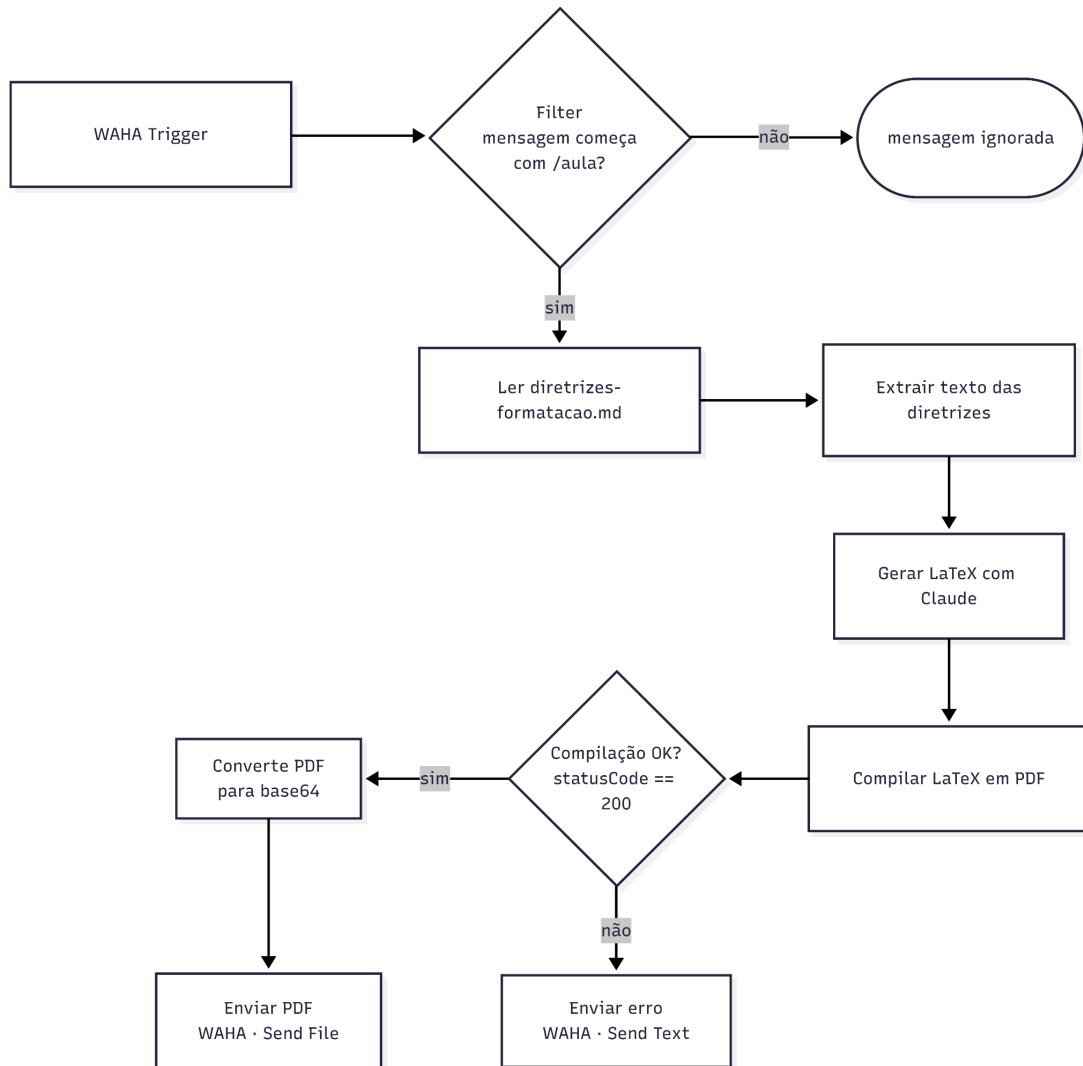
3. Prática: O Gerador de Material de Aula

O fluxo da prática foi montado através da *API REST* do n8n, utilizando *WAHA*, as credenciais da *Anthropic* e um compilador *LaTeX*. O fluxo funciona com o nó *WAHA Trigger* recebendo uma mensagem e dando início ao fluxo, logo na entrada um filtro verifica se o comando */aula* está presente na mensagem de solicitação; um nó de leitura de arquivo carrega as diretrizes de formatação de um arquivo de texto caso o comando seja identificado e o fluxo de fato inicie. Com essas diretrizes o Claude gera um código *LaTeX* completo contendo um resumo sobre o assunto solicitado. O serviço de compilação (*TeX Live + FastAPI*) transforma o código *LaTeX* em um arquivo PDF devidamente formatado. Por fim, o PDF é convertido para base64 e enviado de volta ao usuário pelo WhatsApp.

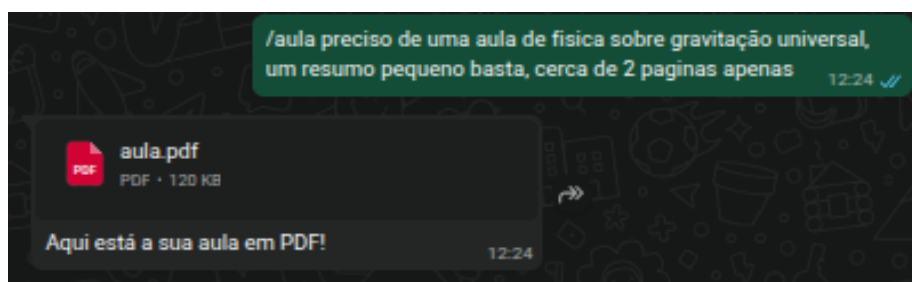
Em caso de falha de alguma dessas etapas, uma mensagem de erro é enviada para notificar o usuário do que ocorreu. Abaixo está representada a arquitetura da infraestrutura implementada.



Fastcamp de Agentes Inteligentes



Fluxo Interno dos Nós do n8n



Solicitação de Material para o Agente

Abaixo está apresentado parte do resultado da solicitação:

Gravitação Universal

Física – Ensino Médio

6 de agosto de 2026

Objetivos de Aprendizagem

- Compreender a Lei da Gravitação Universal de Newton e sua aplicação.
- Identificar as grandezas envolvidas na força gravitacional entre dois corpos.
- Aplicar a fórmula da gravitação em problemas práticos envolvendo massa e distância.
- Reconhecer a importância histórica e científica da gravitação universal.

Conteúdo Teórico

Introdução

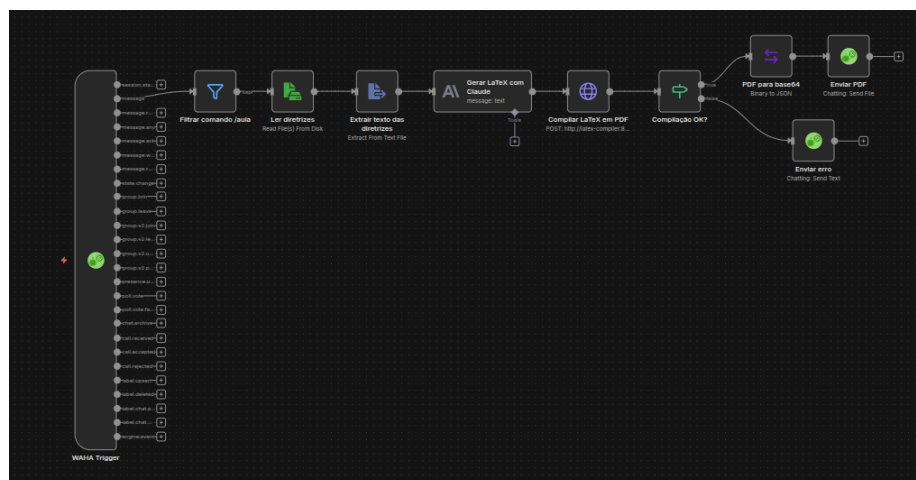
A Gravitação Universal é um dos pilares da Física Clássica, formulada por Isaac Newton no século XVII. Newton percebeu que a mesma força que faz uma maçã cair da árvore é responsável por manter a Lua em órbita ao redor da Terra e os planetas ao redor do Sol. Essa descoberta revolucionou nossa compreensão do universo.

A Lei da Gravitação Universal

A Lei da Gravitação Universal estabelece que dois corpos com massa se atraem mutuamente com uma força diretamente proporcional ao produto de suas massas e inversamente proporcional ao quadrado da distância entre seus centros de massa. Matematicamente, a lei é expressa por:

$$F = G \cdot \frac{m_1 \cdot m_2}{r^2}$$

Resultado em PDF



Fluxo Implementado no n8n

4. Dificuldades Encontradas

4.1 Durante a aula

A engine padrão de conexão do *WAHA* com o *WhatsApp* e a *WEBJS* (baseada em automações de navegador via *whatsapp-web.js*), mostrou-se instável: primeiro apresentando erro 500 ao listar conversas, um bug já conhecido e corrigido com uma atualização de versão da imagem da *WAHA*; em seguida, falhando de forma recorrente ao tentar enviar mensagens de texto (*sendText*). Apesar de documentado, esse segundo bug foi um atraso no desenvolvimento, pois parece não haver uma correção estável até o momento desta implementação. A solução foi migrar para a engine de conexão para *NOWEB*, que implementa o protocolo do *WhatsApp* de forma direta e sem depender de *automações* de navegador. Como a API do *WAHA* é idêntica entre engines, só foi necessário alterar a seguinte linha de código.

```
WHATSAPP_DEFAULT_ENGINE=WEBJS # instável: getChats e sendText falhando  
WHATSAPP_DEFAULT_ENGINE=NOWEB # protocolo direto via Baileys, sem navegador
```

Alterando as Engines do WAHA

Mesmo após a migração de engine, o nó de envio de texto continuava retornando sucesso sem que a mensagem chegasse de fato ao destinatário. Uma inspeção detalhada do JSON do workflow revelou a causa desse problema, o campo *chatId* do nó de envio de texto (*Send Text*) estava configurado incorretamente para usar a ID da mensagem recebida (*id_mensagem*) em vez do identificador do contato de destino (*quem_mandou*), fazendo com que o *WAHA* tentasse repetidamente resolver uma string de ID de mensagem como se fosse um número de telefone de destino. Após corrigir essa referência, a entrega passou a ocorrer de forma consistente em todos os testes subsequentes, inclusive em uma validação externa feita por terceiros a partir de um número diferente.

```
// nó "Send a text message" – antes  
chatId: {{ $('Edit Fields').item.json.id_mensagem }}  
  
// nó "Send a text message" – depois  
chatId: {{ $('Edit Fields').item.json.quem_mandou }}
```

Motivo do n8n Retornar Sucesso sem Entregar a Mensagem

4.2 Durante a prática

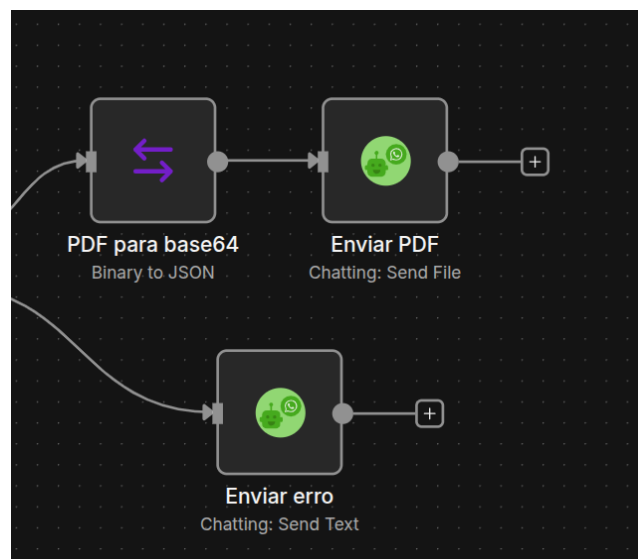
O primeiro problema durante o desenvolvimento dessa etapa foi que mesmo instruído explicitamente a não utilizar determinadas formatações, o Claude “empacotou” o código *LaTeX* dentro de um bloco *markdown*, causando o travamento do *pdflatex* logo no início da execução, tentando interpretar a formatação incoerente em *markdown* (três crases) como comando.

Uma vez percebendo que apenas a instrução do prompt não seria suficiente para evitar esse tipo de problema com o modelo LLM, foi necessário que o nó de compilação do PDF fosse configurado para retirar ativamente os blocos de formatação em *markdown* antes de enviar o conteúdo ao compilador, contornando possíveis problemas de interpretação do modelo.

```
tex = $json.content[0].text
.replace(/^```(latex)?\s*\n?/, '')
.replace(/\n?```\s*$/, '')
```

Trecho de Código que Retira a Formatação em Markdown

Por fim, mesmo com o PDF devidamente compilado, o envio pelo *WhatsApp* falhava com o erro "*Either 'file.url' or 'file.data' must be specified*". A tentativa inicial de acessar o conteúdo binário diretamente não funcionava para uma string. A solução foi adicionar um nó exclusivo para converter o arquivo binário para base64 antes do envio.



Nó de Conversão Adicionado ao Fim do Fluxo

Conclusões

A implementação desses agentes utilizou os conceitos práticos de orquestração de containers com Docker Compose e integrou serviços via webhooks e API REST, além de reforçar a importância de investigar a causa raiz de um problema a partir dos dados brutos, em vez de confiar apenas nas mensagens de log do *front end*. A comparação entre as duas engines de conexão da WAHA evidenciou um *trade-off* recorrente em bibliotecas que dependem de *scraping* utilizando interfaces de terceiros. E as falhas de formatação/interpretação presentes na parte prática deixam claro que é preciso implementar artifícios que ajudem o modelo a não se perder e entregar o resultado esperado.

Referências

[1] WAHA.DEVLIKE.PRO. **Documentação oficial.**

Disponível em: <<https://waha.devlike.pro/>>

Acesso em 04 de agosto de 2026

[2] GITHUB.COM. **Repositório DEVLIKEAPRO oficial no GitHub.**

Disponível em: <<https://github.com/devlikeapro/waha/issues/486>>

Acesso em 04 de agosto de 2026

[3] GITHUB.COM. **Repositório DEVLIKEAPRO oficial no GitHub.**

Disponível em: <<https://github.com/WhiskeySockets/Baileys/issues/853>>

Acesso em 04 de agosto de 2026

[4] GITHUB.COM. **Repositório DEVLIKEAPRO oficial no GitHub.**

Disponível em: <<https://github.com/devlikeapro/n8n-nodes-waha>>

Acesso em 04 de agosto de 2026

[5] FASTAPI.TIANGOLO.COM. **Documentação oficial**

Disponível em: <<https://fastapi.tiangolo.com/>>

Acesso em 05 de agosto de 2026

[6] TUG.ORH. **TeX Live.**

Disponível em: <<https://tug.org/texlive/>>

Acesso em 05 de agosto de 2026